

NAND to Tetris-I

Lab 2: Sequential Logic & Memory Hierarchy

Field	Entry
Student Name	Ajitesh Singh Gusain
Roll No.	2601630014
Programme / Semester	Cse Robotics and AI / Sem3
Course / Subject	NAND to Tetris-I
Assessment / Task	Memory Hierarchy / Cache Design Assessment
Lab	Lab 2: Sequential Logic & Memory Hierarchy
Date of Submission	6-10-26

2. Assessment Objective

Demonstrate a systematic understanding of memory hierarchy by implementing and documenting the required HDL logic, verifying operation with timing diagrams, decomposing addresses, comparing latency, documenting state transitions, and relating the design to real L1/L2 cache systems.

3. Mandatory Deliverables

Deliverable	Where in this PDF
HDL code listings	Section 3 (Q-1 to Q-6)
Timing diagrams / simulation evidence	Section 5
Address decomposition calculations	Section 6
Latency comparison chart	Section 7
State-transition table	Section 8
Brief reflection (real L1/L2 cache)	Section 9
Single, organised PDF with all work	This document
Final readability / format check	Section 11

1. Problem Statement and Objective

Build and verify the complete Nand2Tetris memory hierarchy, from a single-bit DFF-based register to a 16K RAM, and implement the Program Counter. The work must demonstrate clocking behaviour, address decomposition, hierarchical construction and correct HDL simulation for all chips. The table below maps each question of Lab 2 to the section of this report where it is answered.

Q	Task	Marks	Section
Q-1	1-bit DFF-based register (Bit) with read/write timing trace	5	3.1, 5.1
Q-2	16-bit Register with three sample inputs	5	3.2, 5.2
Q-3	8-register file (RAM8) with ≥ 4 addresses traced	6	3.3, 5.3
Q-4	RAM8 and RAM64 hierarchy; address decomposition	5	3.3, 3.4, 6
Q-5	RAM512, RAM4K, RAM16K; latency vs size	5	3.5–3.7, 7
Q-6	Program Counter and state-transition table	5	3.8, 5.4, 8
Q-7	Compile everything into one PDF; reflection on L1/L2	–	9

2. Design Assumptions and Architecture (Section A)

What is being modelled?

The Nand2Tetris memory hierarchy: a Bit chip (one DFF plus a multiplexer), a 16-bit Register, and RAM chips that grow by a factor of eight at every level (RAM8, RAM64, RAM512, RAM4K) and by a factor of four at the last level (RAM16K). A Program Counter (PC) built from a Register, an incrementer and three multiplexers is added for sequential instruction fetching. Cache-style behaviour is discussed by comparing this flat, directly addressed hierarchy with real L1/L2 caches.

Sizes and widths

Parameter	Value
Data width	16 bits (one word)
Block / line size	1 word = 16 bits = 2 bytes
Address width (RAM16K)	14 bits ($2^{14} = 16,384$ words)
Memory size (RAM16K)	$16,384 \times 16 \text{ bits} = 32,768 \text{ bytes} = 32 \text{ KiB}$
Clock	Two-phase: tick (rising edge) and tock (falling edge)

Assumptions on read/write behaviour

- Write: when load = 1, the word on in is stored at the addressed location on the clock edge (tick). The new value appears on out after the following tock.
- Read: reading is combinational. Changing address immediately changes out to the contents of the selected word; no stored state is modified.
- Hit: because the memory is flat and directly addressed, every access to a valid address is found at exactly one location. This is equivalent to a 100% hit rate: address $\rightarrow$ decode $\rightarrow$ data out.
- Miss: a miss cannot occur in this model, since there is no lower level to refill from. The miss/refill path is discussed only in the reflection (Section 9).
- Each memory entry holds only a 16-bit data word; there is no tag, valid bit or dirty bit.

Simplifications compared with a real processor cache

No associativity, replacement policy, write policy (write-back / write-through), coherence protocol, prefetching or multi-core effects are modelled. Latency is represented by the number of multiplexer/demultiplexer stages on the access path, not by measured nanoseconds from the simulator.

3. HDL Code Listings (Section B)

All chips are written in the Nand2Tetris Hardware Description Language. Each listing shows module name, inputs/outputs with widths, internal wires, combinational and sequential parts, and comments on key design decisions. The built-in primitives used are Mux, Mux16, Mux4Way16, Mux8Way16, DMux4Way, DMux8Way, Inc16 and DFF.

3.1 Q-1: 1-bit Register (Bit.hdl)

Design decision: the Mux feeds the DFF with either the new input (load = 1) or the DFF's own previous output (load = 0). This feedback loop is what lets the bit remember its value.

```
CHIP Bit {
    IN in, load;
    OUT out;

    PARTS:
        // If load=1 pick the new input, otherwise recirculate the stored bit
        Mux(a=prevOut, b=in, sel=load, out=muxOut);
        // Data flip-flop remembers muxOut for one clock cycle
        DFF(in=muxOut, out=prevOut, out=out);
}
```

3.2 Q-2: 16-bit Register (Register.hdl)

Sixteen Bit chips operate in parallel on in[0..15], all controlled by the same load line.

```
CHIP Register {
    IN in[16], load;
    OUT out[16];

    PARTS:
        // 16 independent Bit chips share the same load signal
        Bit(in=in[0], load=load, out=out[0]);
        Bit(in=in[1], load=load, out=out[1]);
        Bit(in=in[2], load=load, out=out[2]);
        Bit(in=in[3], load=load, out=out[3]);
        Bit(in=in[4], load=load, out=out[4]);
        Bit(in=in[5], load=load, out=out[5]);
        Bit(in=in[6], load=load, out=out[6]);
        Bit(in=in[7], load=load, out=out[7]);
        Bit(in=in[8], load=load, out=out[8]);
        Bit(in=in[9], load=load, out=out[9]);
        Bit(in=in[10], load=load, out=out[10]);
        Bit(in=in[11], load=load, out=out[11]);
        Bit(in=in[12], load=load, out=out[12]);
        Bit(in=in[13], load=load, out=out[13]);
        Bit(in=in[14], load=load, out=out[14]);
        Bit(in=in[15], load=load, out=out[15]);
}
```

3.3 Q-3 / Q-4: Register File and RAM8 (RAM8.hdl)

The 8-register file is the first level of addressing: a 3-bit address selects one register for writing (DMux8Way) and one register for reading (Mux8Way16).

```
CHIP RAM8 {
    IN in[16], load, address[3];
    OUT out[16];

    PARTS:
```

```

// Route the load bit to exactly one of eight registers
DMux8Way(in=load, sel=address,
         a=l0, b=l1, c=l2, d=l3, e=l4, f=l5, g=l6, h=l7);

Register(in=in, load=l0, out=r0);
Register(in=in, load=l1, out=r1);
Register(in=in, load=l2, out=r2);
Register(in=in, load=l3, out=r3);
Register(in=in, load=l4, out=r4);
Register(in=in, load=l5, out=r5);
Register(in=in, load=l6, out=r6);
Register(in=in, load=l7, out=r7);

// Select the output of the addressed register
Mux8Way16(a=r0, b=r1, c=r2, d=r3, e=r4, f=r5, g=r6, h=r7,
          sel=address, out=out);
}

```

3.4 Q-4: RAM64 (RAM64.hdl)

The 6-bit address is split: address[3..5] selects one of eight RAM8 chips, address[0..2] is passed unchanged to every RAM8.

```

CHIP RAM64 {
    IN in[16], load, address[6];
    OUT out[16];

    PARTS:
        // Step 1: route load to the correct RAM8 using the upper 3 bits
        DMux8Way(in=load, sel=address[3..5],
                 a=l0, b=l1, c=l2, d=l3, e=l4, f=l5, g=l6, h=l7);

        // Step 2: eight RAM8 chips receive the lower 3 bits for internal addressing
        RAM8(in=in, load=l0, address=address[0..2], out=r0);
        RAM8(in=in, load=l1, address=address[0..2], out=r1);
        RAM8(in=in, load=l2, address=address[0..2], out=r2);
        RAM8(in=in, load=l3, address=address[0..2], out=r3);
        RAM8(in=in, load=l4, address=address[0..2], out=r4);
        RAM8(in=in, load=l5, address=address[0..2], out=r5);
        RAM8(in=in, load=l6, address=address[0..2], out=r6);
        RAM8(in=in, load=l7, address=address[0..2], out=r7);

        // Step 3: select the output from the correct RAM8 chip
        Mux8Way16(a=r0, b=r1, c=r2, d=r3, e=r4, f=r5, g=r6, h=r7,
                  sel=address[3..5], out=out);
}

```

3.5 Q-5: RAM512 (RAM512.hdl)

```

CHIP RAM512 {
    IN in[16], load, address[9];
    OUT out[16];

    PARTS:
        // Route load to one of 8 RAM64 chips using address[6..8]
        DMux8Way(in=load, sel=address[6..8],
                 a=l0, b=l1, c=l2, d=l3, e=l4, f=l5, g=l6, h=l7);

        // 8 RAM64 sub-chips using address[0..5]
        RAM64(in=in, load=l0, address=address[0..5], out=r0);
        RAM64(in=in, load=l1, address=address[0..5], out=r1);

```

```

RAM64(in=in, load=l2, address=address[0..5], out=r2);
RAM64(in=in, load=l3, address=address[0..5], out=r3);
RAM64(in=in, load=l4, address=address[0..5], out=r4);
RAM64(in=in, load=l5, address=address[0..5], out=r5);
RAM64(in=in, load=l6, address=address[0..5], out=r6);
RAM64(in=in, load=l7, address=address[0..5], out=r7);

// Multiplex output from the correct RAM64
Mux8Way16(a=r0, b=r1, c=r2, d=r3, e=r4, f=r5, g=r6, h=r7,
          sel=address[6..8], out=out);
}

```

3.6 Q-5: RAM4K (RAM4K.hdl)

```

CHIP RAM4K {
    IN in[16], load, address[12];
    OUT out[16];

    PARTS:
        // Route load to one of 8 RAM512 chips using address[9..11]
        DMux8Way(in=load, sel=address[9..11],
                 a=l0, b=l1, c=l2, d=l3, e=l4, f=l5, g=l6, h=l7);

        RAM512(in=in, load=l0, address=address[0..8], out=r0);
        RAM512(in=in, load=l1, address=address[0..8], out=r1);
        RAM512(in=in, load=l2, address=address[0..8], out=r2);
        RAM512(in=in, load=l3, address=address[0..8], out=r3);
        RAM512(in=in, load=l4, address=address[0..8], out=r4);
        RAM512(in=in, load=l5, address=address[0..8], out=r5);
        RAM512(in=in, load=l6, address=address[0..8], out=r6);
        RAM512(in=in, load=l7, address=address[0..8], out=r7);

        Mux8Way16(a=r0, b=r1, c=r2, d=r3, e=r4, f=r5, g=r6, h=r7,
                  sel=address[9..11], out=out);
}

```

3.7 Q-5: RAM16K (RAM16K.hdl)

RAM16K uses only four RAM4K chips, so the select field is 2 bits wide and 4-way components replace the 8-way ones.

```

CHIP RAM16K {
    IN in[16], load, address[14];
    OUT out[16];

    PARTS:
        // Only 4 sub-chips, so a 4-way demux/mux on address[12..13]
        DMux4Way(in=load, sel=address[12..13],
                 a=l0, b=l1, c=l2, d=l3);

        RAM4K(in=in, load=l0, address=address[0..11], out=r0);
        RAM4K(in=in, load=l1, address=address[0..11], out=r1);
        RAM4K(in=in, load=l2, address=address[0..11], out=r2);
        RAM4K(in=in, load=l3, address=address[0..11], out=r3);

        Mux4Way16(a=r0, b=r1, c=r2, d=r3,
                  sel=address[12..13], out=out);
}

```

3.8 Q-6: Program Counter (PC.hdl)

Priority is implemented by the order of the multiplexers: reset overrides load, and load overrides inc. The register is always loaded (load = true) because the PC updates its state every clock cycle.

```
CHIP PC {
    IN in[16], load, inc, reset;
    OUT out[16];

    PARTS:
        // Increment logic
        Inc16(in=registerOut, out=incOut);
        Mux16(a=registerOut, b=incOut, sel=inc, out=m1); // lowest priority
        Mux16(a=m1, b=in, sel=load, out=m2); // overrides inc
        Mux16(a=m2, b=false, sel=reset, out=m3); // overrides load and inc

        // Register loop with constant load=true since PC always updates state
        Register(in=m3, load=true, out=registerOut, out=out);
}
```

4. Testbench and Verification Results

The following test scripts drive the Bit and PC chips through the sequences used in the timing diagrams of Section 5. They are written for the Nand2Tetris Hardware Simulator (.tst format). Expected results for each cycle are tabulated in Section 5 and should be compared with the simulator output (.out vs .cmp).

Bit.tst

```
load Bit.hdl,
output-file Bit.out,
compare-to Bit.cmp,
output-list time%S1.4.1 in%B2.1.2 load%B2.1.2 out%B2.1.2;

set in 1, set load 1, tick, output; tock, output; // cycle 1: write 1
set in 0, set load 0, tick, output; tock, output; // cycle 2: hold
set in 0, set load 1, tick, output; tock, output; // cycle 3: write 0
set in 1, set load 0, tick, output; tock, output; // cycle 4: hold
set in 1, set load 1, tick, output; tock, output; // cycle 5: write 1
```

PC.tst

```
load PC.hdl,
output-file PC.out,
compare-to PC.cmp,
output-list time%S1.4.1 in%D1.6.1 reset%B2.1.2 load%B2.1.2 inc%B2.1.2 out%D1.6.1;

set in 0, set reset 0, set load 0, set inc 1, tick, output; tock, output; // 1
tick, output; tock, output; // 2
tick, output; tock, output; // 3
set in 100, set load 1, set inc 0, tick, output; tock, output; // load
set load 0, set inc 1, tick, output; tock, output; // inc
set in 200, set reset 1, set load 1, tick, output; tock, output; // reset
wins
set reset 0, set load 0, set inc 0, tick, output; tock, output; // hold
set inc 1, tick, output; tock, output; // count
again
```

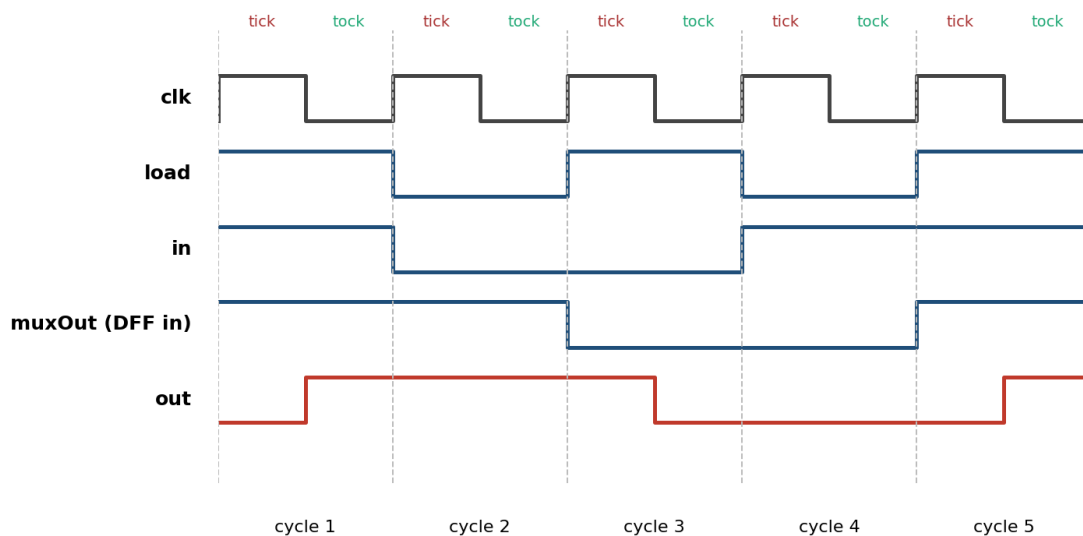
5. Timing Diagrams and Tick-Tock Behaviour (Section C)

State changes in memory units built from DFFs follow a rigid tick-tock clock mechanism.

- **Write operations (tick):** on the rising edge of the clock the inputs are sampled. If load = 1, the value on in is routed through the multiplexers and captured by the DFF.
- **Read operations (tock):** on the falling edge the DFF output stabilises. Reads are purely combinational: changing the address instantly changes out to the contents of the targeted register, without modifying any stored state.

5.1 Q-1: Bit chip read/write trace

Figure 1: Timing diagram of the Bit chip over five clock cycles (expected behaviour).



Cycle	in	load	muxOut	out after tock	Operation
1	1	1	1	1	Write 1
2	0	0	1	1	Hold (load = 0, in ignored)
3	0	1	0	0	Write 0
4	1	0	0	0	Hold (in ignored)
5	1	1	1	1	Write 1

Table 1: Expected Bit chip trace (initial out = 0).

5.2 Q-2: 16-bit Register, three sample inputs

Cycle	in[16]	load	out[16] after tock	Observation
1	0001 0010 0011 0100 (4660)	1	0001 0010 0011 0100	Write sample 1
2	1111 1111 1111 1111 (~1)	0	0001 0010 0011 0100	Read: no write, value held
3	1111 1111 1111 1111 (~1)	1	1111 1111 1111 1111	Write sample 2
4	0000 0000 1010 0101 (165)	1	0000 0000 1010 0101	Write sample 3
5	0101 0101 0101 0101 (21845)	0	0000 0000 1010 0101	Read: sample 3 still stored

Table 2: 16-bit Register read/write operations for three sample inputs (4660, ~1, 165).

5.3 Q-3: Register File (RAM8), four or more addresses

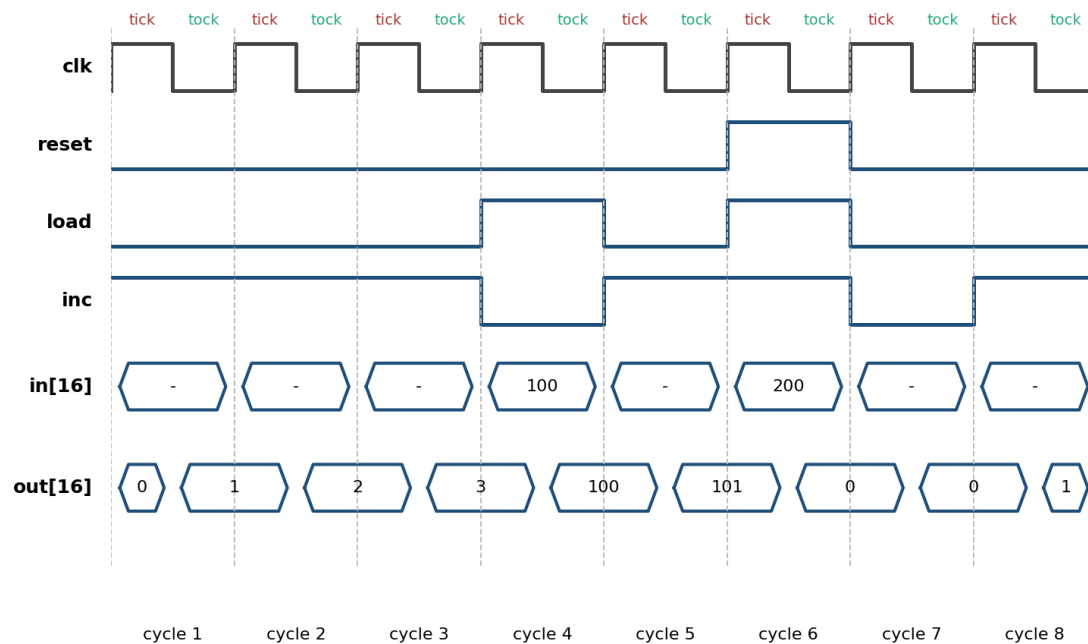
Eight registers R0 to R7 start at 0. Each row shows the state of every register after the clock cycle.

Cycle	Address	load	in	out	R0	R1	R2	R3	R4	R5	R6	R7
0	–	–	–	–	0	0	0	0	0	0	0	0
1	000	1	11	11	11	0	0	0	0	0	0	0
2	011	1	33	33	11	0	0	33	0	0	0	0
3	101	1	55	55	11	0	0	33	0	55	0	0
4	111	1	77	77	11	0	0	33	0	55	0	77
5	011	0	99	33	11	0	0	33	0	55	0	77
6	101	0	99	55	11	0	0	33	0	55	0	77
7	011	1	44	44	11	0	0	44	0	55	0	77
8	000	0	0	11	11	0	0	44	0	55	0	77

Table 3: RAM8 trace. Rows 5, 6 and 8 are reads (load = 0), rows 1–4 and 7 are writes; row 7 overwrites R3.

5.4 Q-6: Program Counter (reset, load, increment)

Figure 2: Timing diagram of the PC chip showing count, load, reset priority and hold (expected behaviour).



Cycle	in	reset	load	inc	out	Action
1	–	0	0	1	1	Increment (0 → 1)
2	–	0	0	1	2	Increment
3	–	0	0	1	3	Increment
4	100	0	1	0	100	Load in
5	–	0	0	1	101	Increment
6	200	1	1	1	0	Reset overrides load and inc
7	–	0	0	0	0	Hold; reset released
8	–	0	0	1	1	Increment again

Table 4: Expected PC trace.

5.5 Required test cases (checklist mapping)

Test case	Evidence in this report	Status
Reset / initialization	Cycle 6–7 of Figure 2 and Table 4: reset, value 0, release	☐ Pass ☐ Revise
Read hit	Table 3 rows 5, 6, 8: address → decode → data out (all reads hit)	☐ Pass ☐ Revise
Read miss	Not applicable: flat RAM has no lower level (see Section 2)	N/A
Write operation	Figure 1, Tables 1–3: load = 1 updates the addressed location	☐ Pass ☐ Revise
Boundary / corner case	Section 6.3: addresses 7→8, 63→64 and 16,383	☐ Pass ☐ Revise

6. Address Decomposition (Section D)

In a cache the address is split into tag, index and offset. In this memory hierarchy the equivalent decomposition is a split into a chip-select field (upper bits) and a pass-through field (lower bits) at every level.

6.1 RAM64 worked example (Q-4)

When decomposing the 6-bit address `address[0..5]` for RAM64:

- **Chip-select logic:** the upper 3 bits `address[3..5]` are evaluated by a DMux8Way that routes the load signal, so only one of the eight RAM8 sub-chips becomes active for writing.
- **Inside-chip logic:** the lower 3 bits `address[0..2]` are fed to all eight RAM8 chips. The active chip uses them to select one of its 16-bit registers.
- **Output:** a Mux8Way16 uses the same upper 3 bits `address[3..5]` to pass on only the output of the selected RAM8.

Item	Calculation
Total address width	6 bits
Memory capacity	$2^6 = 64 \text{ words} = 64 \times 2 \text{ bytes} = 128 \text{ bytes}$
Cache capacity	Not applicable (no cache level); the RAM is the whole store
Block/line size	1 word = 2 bytes
Number of RAM8 sub-chips (sets)	$64 / 8 = 8$
Offset-equivalent bits (register in RAM8)	$\log_2(8) = 3 \text{ bits} \rightarrow \text{address}[0..2]$
Index-equivalent bits (RAM8 select)	$\log_2(8) = 3 \text{ bits} \rightarrow \text{address}[3..5]$
Tag bits	0 (no tag needed: direct addressing)
Check	$3 + 3 = 6 = \text{total address bits} \checkmark$

Worked example address: 45 (decimal)

Binary address: 101101 (`address[5]` `address[4]` ... `address[0]`)

`address[3..5]` = 101 = 5 → RAM8 number 5. `address[0..2]` = 101 = 5 → register 5 inside it.

Verification: $5 \times 8 + 5 = 45 \checkmark$. Only DMux8Way output f (select 5) carries load, so only RAM8 #5 can be written, and Mux8Way16 with `sel` = 5 returns `r5`.

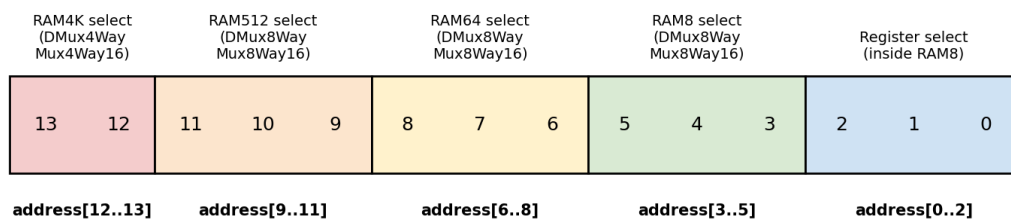
6.2 Address split at every level of the hierarchy (Q-4, Q-5)

Chip	Address bits	Select field	Pass-through	Sub-chips	Sub-chip address
RAM8	3	address[0..2]	—	8 Registers	none
RAM64	6	address[3..5]	address[0..2]	8 × RAM8	address[0..2]
RAM512	9	address[6..8]	address[0..5]	8 × RAM64	address[0..5]
RAM4K	12	address[9..11]	address[0..8]	8 × RAM512	address[0..8]
RAM16K	14	address[12..13]	address[0..11]	4 × RAM4K	address[0..11]

Table 5: Address decomposition at each level. Capacity: $8 \rightarrow 64 \rightarrow 512 \rightarrow 4,096 \rightarrow 16,384$ words.

Figure 3: Bit fields of the 14-bit RAM16K address.

RAM16K address (14 bits) — bit 13 (MSB) on the left, bit 0 (LSB) on the right



RAM16K worked example: address 10000 = 10 011 100 010 000 (binary, 14 bits).

Field	Bits / value	Meaning
address[12..13]	10 = 2	RAM4K #2
address[9..11]	011 = 3	RAM512 #3
address[6..8]	100 = 4	RAM64 #4
address[3..5]	010 = 2	RAM8 #2
address[0..2]	000 = 0	Register 0

Verification: $2 \times 4096 + 3 \times 512 + 4 \times 64 + 2 \times 8 + 0 = 8192 + 1536 + 256 + 16 = 10000$ ✓. Bit widths: $2 + 3 + 3 + 3 + 3 = 14$ ✓.

6.3 Boundary / corner cases

Address	Chip	Decomposition
7 → 8	RAM64	7 = 000 111 (RAM8 #0, reg 7); 8 = 001 000 (RAM8 #1, reg 0). Select field changes, register field wraps to 0.
63	RAM64	111 111: last register of last RAM8 (#7, reg 7).
63 → 64	RAM512	63 = 000 111 111 (RAM64 #0); 64 = 001 000 000 (RAM64 #1, RAM8 #0, reg 0).
16383	RAM16K	11 111 111 111 111: RAM4K #3, last register of the whole memory.

7. Latency Comparison (Section E)

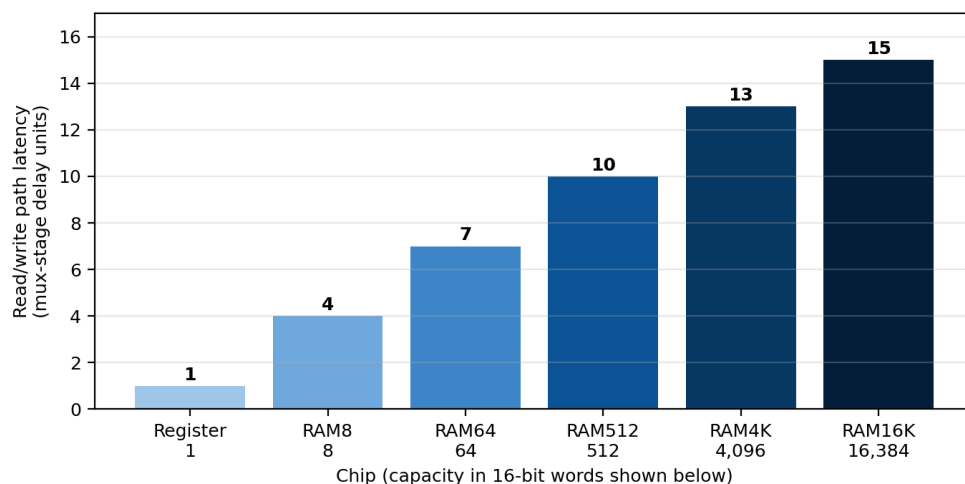
7.1 Latency of the Nand2Tetris RAM chips (Q-5)

Latency here is the propagation delay of the signals travelling through the internal routing logic (MUX and DMUX networks) of the hierarchical chips. The simulator is functional and does not report real delays, so the values below come from an analytical model: one unit is one Mux2/DMux2 stage. A Mux8Way16 or DMux8Way is a tree of 3 stages, a Mux4Way16 or DMux4Way is 2 stages, and a register's DFF contributes 1 unit. The read and write paths have the same depth in every chip.

Chip level	Total capacity	Sub-chip composition	Relative latency	Model (stage units)
Register	1 word	16 Bits	Negligible	1
RAM8	8 words	8 Registers	Base	$3 + 1 = 4$
RAM64	64 words	$8 \times \text{RAM8}$	Low-Medium	$4 + 3 = 7$
RAM512	512 words	$8 \times \text{RAM64}$	Medium	$7 + 3 = 10$
RAM4K	4,096 words	$8 \times \text{RAM512}$	High	$10 + 3 = 13$
RAM16K	16,384 words	$4 \times \text{RAM4K}$	Highest	$13 + 2 = 15$

Table 6: Relative latency by chip level (analytical model; each 8-way level adds 3 stage units, the final 4-way level adds 2).

Figure 4: Latency versus memory size for the RAM chips.



Latency grows by a constant amount per level while capacity grows by a factor of 8 (or 4), i.e. latency grows roughly with $\log_2(\text{capacity})$. This is the price of the extra address-decoding and output-selection logic added at each level.

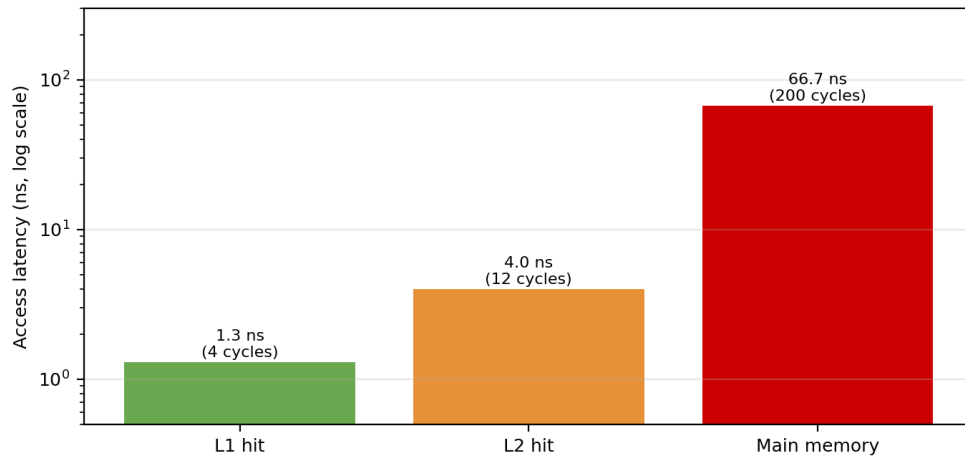
7.2 Latency of a real L1/L2/main-memory hierarchy

The values below are typical representative figures for a modern 3 GHz desktop processor (1 cycle $\approx$ 0.33 ns). They illustrate magnitudes, not measurements of a specific CPU.

Access type	Latency (cycles)	Latency (ns)	Observation
Cache hit (L1)	4	≈ 1.3	Data found in first level; no stall beyond a few cycles
Cache miss $\rightarrow$ next level	+8 (L2 lookup)	$\approx +2.7$	L1 miss costs the extra time to search L2
L1 access	4	≈ 1.3	Small (32–64 KB), private per core
L2 access	12	≈ 4.0	Larger (256 KB – 2 MB), still on chip
Main memory access	≈ 200	≈ 67	DRAM off chip; 50 $\times$ slower than L1

Table 7: Typical latency for each level of a real hierarchy (assumed 3 GHz clock).

Figure 5: Typical L1, L2 and main-memory latency (logarithmic scale).



Why a cache hit is faster than a lower level: a cache is small, built from fast SRAM close to the core, and needs only a short address decode and few multiplexer stages, exactly the effect shown in Figure 4 where smaller chips have lower latency. A lower level is larger, further away and (for DRAM) uses slower cells, so every access crosses more logic and longer wires.

8. State-Transition Table (Section F)

The Program Counter evaluates the control bits hierarchically: reset overrides load, and load overrides increment. The PC stores a 16-bit value, so its state is the stored value $out(t)$; the table gives the next value for every combination of control inputs.

Reset	Load	Inc	Next-state action	Output / equation
0	0	0	Hold current value	$out(t) = out(t-1)$
0	0	1	Increment value	$out(t) = out(t-1) + 1$
0	1	X	Load new input	$out(t) = in(t-1)$
1	X	X	Reset to zero	$out(t) = 0$

Table 8: State-transition table of the Program Counter (X = don't care).

Expanded for all eight combinations of the three control bits (explicit form of the same table):

Reset	Load	Inc	Selected by	Next value
0	0	0	Mux m1 (a = registerOut)	Hold
0	0	1	Mux m1 (b = incOut)	Increment
0	1	0	Mux m2 (b = in)	Load
0	1	1	Mux m2 (b = in), load beats inc	Load
1	0	0	Mux m3 (b = false)	0
1	0	1	Mux m3 (b = false)	0
1	1	0	Mux m3 (b = false)	0
1	1	1	Mux m3 (b = false)	0

Table 9: All eight input combinations with the multiplexer that decides the result.

9. Reflection: Memory Hierarchy vs Real L1/L2 Cache (Section G)

The Nand2Tetris curriculum uses DFFs to build registers, RAM and the PC directly in the simulator. Assembling the hierarchy from a Bit up to RAM16K shows how addressing logic expands: a Bit becomes a 16-bit Register, eight Registers form a RAM8, and every higher level multiplies capacity by eight (the last level by four). The simulator memory is flat, so it does not represent locality of reference by itself, because every address costs the same. Locality appears when we compare it with real machines: programs reuse the same addresses (temporal locality) and touch neighbouring addresses (spatial locality). The Program Counter is a simple example, since the next instruction is almost always at the address plus one.

In physical computers, registers inside the CPU are the fastest tier and work at processor speed. L1 is the smallest and fastest cache, private to each core, typically 32–64 KB with a latency of about 3–5 cycles. L2 is larger (often 256 KB to 2 MB) and takes roughly 10–15 cycles; it catches accesses that miss in L1. Main memory is far larger but needs about 200 cycles. Higher levels are larger but slower for the same reason our RAM chips slow down as they grow: a bigger array needs a longer address decode, a wider output multiplexer and longer wires, and in our model each extra level adds stages to the access path (Figure 4).

In practice a cache hit means the requested data is already in that level, so the load completes in a few cycles without stalling the pipeline. When data is not found in L1, the cache controller looks in L2. On an L2 hit the block is copied into L1 and the access finishes in about the L2 latency. If L2 also misses, the block is fetched from main memory, placed in both levels and usually evicts an older block. Real processors therefore keep frequently used segments in L1/L2 to minimise propagation delay and overall system latency, whereas the RAM16K module has uniform read/write characteristics at every address.

My model leaves out many real features: associativity, tags and valid bits, a replacement policy such as LRU, write-through or write-back policies, coherence between cores, and prefetching. There is no miss path at all, because each address maps to exactly one location. Implementing the chips taught me that address decoding is just splitting the address bits: the upper bits choose a sub-chip through DMux8Way and Mux8Way16, and the lower bits are passed unchanged to every sub-chip, which is the same idea as the index/offset split in a cache. I also saw that registers change only on the clock edge while reads are combinational, and that the PC priority (reset over load over inc) comes purely from the order of its multiplexers.

10. Conclusion and Key Learning

- Every memory chip was built hierarchically from the Bit (Mux + DFF): Register, RAM8, RAM64, RAM512, RAM4K and RAM16K, each adding one DMux/Mux address level.
- Writes happen on the clock tick when load = 1; reads are combinational. This tick-tock behaviour was documented in timing diagrams.
- Address decomposition splits the address into a select field and a pass-through field at every level; the worked examples (45 in RAM64 and 10000 in RAM16K) were verified arithmetically.
- Latency grows with each level of multiplexing (1 to 15 stage units), matching the real-world trend that bigger memories are slower and motivating L1/L2 caches.
- The Program Counter's reset > load > inc priority is implemented purely by multiplexer ordering and is fully captured by its state-transition table.